TP – Stockage Temporaire des Données dans Android

BTS DAI – Développement Mobile

Objectifs du TP

À la fin de ce TP, l'étudiant sera capable de :
— Comprendre la différence entre stockage persistant et stockage temporaire
— Utiliser la mémoire volatile (RAM)
— Mettre en place un système de cache
— Gérer une session utilisateur dans une application Android
— Afficher des données temporaires dans une interface utilisant **StackView**

Introduction

Dans une application mobile, certaines données ne doivent pas forcément être sauvegardées de manière permanente dans une base de données.

On parle alors de **stockage temporaire**.

Ce stockage permet :
— d'améliorer les performances
— de réduire les accès réseau
— d'accélérer l'affichage des données

Dans ce TP nous allons étudier trois mécanismes :
— le **Cache**
— la **Mémoire volatile**
— la **Session utilisateur**

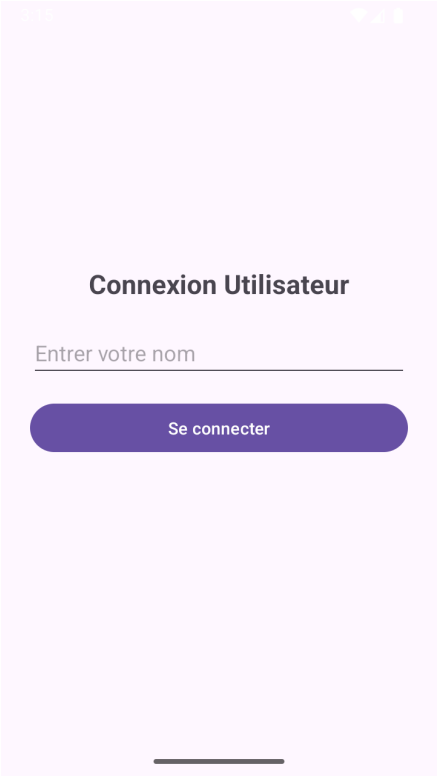
L'objectif de ce TP est de créer une mini application contenant :
— un écran de connexion
— un écran affichant des produits
— les produits sont stockés dans un cache
— l'utilisateur connecté est conservé dans une session
— les produits sont affichés dans un StackView

1 Partie 1 – Mémoire Volatile (RAM)

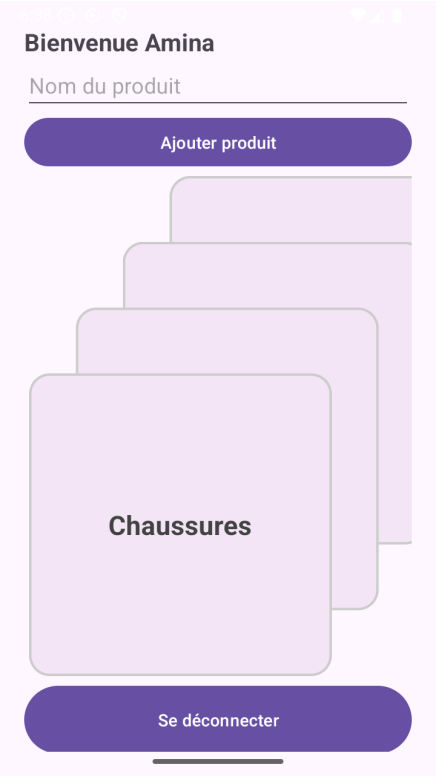
1.1 Principe

La mémoire volatile correspond aux variables stockées en RAM pendant l'exécution de l'application.

Caractéristiques :
— Très rapide
— Disponible uniquement pendant l'exécution
— Les données sont perdues lorsque l'application est fermée



(a) Liste StackView contenant les données



(b) Interface Login

FIGURE 1 – Interfaces de l'application

1.2 Exemple Théorique

Une variable Java peut être utilisée pour stocker temporairement une information.

2 Partie 4 – Session Utilisateur

2.1 Principe

Une session permet de conserver les informations d'un utilisateur pendant l'utilisation de l'application.

Exemples :

— utilisateur connecté
— panier d'achat
— préférences temporaires

La session est généralement stockée :

— en RAM
— dans une classe singleton

2.2 Exemple Théorique

```
public class SessionManager {

    private static String utilisateur;
```

```
    public static void setUtilisateur(String u){
        utilisateur = u;
    }

    public static String getUtilisateur(){
        return utilisateur;
    }

    public static void logout(){
        utilisateur = null;
    }

}
```

Utilisation :

```
SessionManager.utilisateurConnecte = "Ahmed";
SessionManager.setUtilisateur(nom);
String u = SessionManager.getUser();
```

2.3 Travail demandé

Créer une application contenant :
— un champ EditText pour saisir le nom de l'utilisateur
— l'utilisateur saisit son nom, et clique sur un bouton "Se connecter"
— un TextView affichant l'utilisateur connecté
—
— le nom de l'utilisateur apparaît sur toutes les activités

Consignes
— créer une classe **SessionManager**
— stocker le nom de l'utilisateur dans une variable statique
— afficher l'utilisateur connecté dans l'activité principale

2.4 Partie pratique

Compléter la classe suivante :

```
public class SessionManager {

    private  ______ String utilisateur;

    public static void setUtilisateur(String u){
        ______ = u;
    }

    public static String getUtilisateur(){
        return ______;
    }
    public static void logout(){
```

```
        ------;
    }

}
```

3 Partie 2 – Le Cache

3.1 Principe

Le cache permet de stocker temporairement des données afin d'éviter de les recalculer ou de les télécharger plusieurs fois.

Exemples :

— images téléchargées depuis internet
— informations produits
— résultats d'une API

Android propose une classe appelée :

LruCache

Elle supprime automatiquement les données les moins utilisées.

3.2 Exemple Théorique

Création d'un cache :

```
int cacheSize = 4 * 1024 * 1024;

LruCache<String, String> cache =
        new LruCache<>(cacheSize);
```

Ajouter une donnée :

```
cache.put("produit1", "Chaussures␣Nike");
```

Lire une donnée :

```
String produit = cache.get("produit1");
```

3.3 Travail demandé

Créer un système de cache pour stocker des produits.

Chaque produit possède :

— un identifiant
— un nom

Étapes

— créer un cache avec LruCache
— ajouter 5 produits dans le cache
— récupérer un produit par son identifiant

```java
public class CacheManager {

    public static ______<String,String> cache;

      public static ArrayList<String> produits = new ArrayList<>();

    public static void initCache(){
     if(cache == null){

        int cacheSize = 4 * 1024 * 1024;

        cache = new ______<>(cacheSize);

        cache.put("1","________");
        cache.put("2","________");
        cache.put("3","________");

    }}

}
```

4 Partie 3 : Connexion utilisateur

4.1 Partie théorique

Dans l'activité principale (MainActivity.java), nous récupérons le nom de l'utilisateur et nous l'enregistrons dans la session. Il faut commencer d'abord par initier le cache via la méthode initCache qu'on a crée dans la classe CacheManager.

Exemple :

```java
CacheManager.initCache();

btnLogin.setOnClickListener(v -> {

    String nom = editNom.getText().toString();

    SessionManager.setUtilisateur(nom);

    Intent i = new Intent(MainActivity.this,
            ProduitsActivity.class);

    startActivity(i);

});
```

5 Partie 4 – Adapter personnalisé

Le StackView a besoin d'un **Adapter** pour afficher les données.

5.1 Travail demandé

Créer la classe **ProduitAdapter**, contenant une arraylist : ArrayList<String> produits ; et les méthode : getCount, getItem, getItemId et getView. Afficher les produits stockés dans le cache dans un StackView, via getView .

```
public ProduitAdapter(________, ________) {
        this.________ = ________;
        this.________ = ________;
        inflater = LayoutInflater.________;
    }
    @Override
    public int getCount() {
        return produits.size();
    }

    @Override
    public Object getItem(int i) {
        return produits.get(i);
    }

    @Override
    public long getItemId(int i) {
        return i;
    }
    @Override
    public View getView(int i, View view, ViewGroup parent) {

        LayoutInflater ________ = ________;
        view = ________;

        TextView txtProduit = view.________(R.id.txtProduit);

        txtProduit.________(produits.________);

        return ________;
    }
```

6 Partie 5 – Affichage des données avec StackView

Pour afficher les produits, nous allons utiliser un composant Android appelé :
StackView
Il permet d'afficher les éléments comme une pile de cartes.

6.1 activity_produits.xml

```
<StackView
    android:id="@+id/stackView"
    android:layout_width="match_parent"
```

```
    android:layout_height="300dp"/>
```

Ajouter également un textView pour afficher un message de bienvenue, selon le nom de l'utilisateur. Un EditText pour ajouter de nouveaux produits, et un bouton pour activer l'insertion. A la fin, ajouter un bouton de déconnexion. Dans le dossier Drawable, créer un nouveau fichier nommé **card_background**.

```
    <shape xmlns:android="http://schemas.android.com/apk/res/
       android">

    <!-- couleur de fond -->
    <solid android:color="#F3E5F5"/>

    <!-- coins arrondis -->
    <corners android:radius="16dp"/>

    <!-- bordure -->
    <stroke
        android:width="2dp"
        android:color="#CCCCCC"/>

    <!-- espace interne -->
    <padding
        android:left="12dp"
        android:top="12dp"
        android:right="12dp"
        android:bottom="12dp"/>

</shape>
```

6.2 item_produit.xml

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:gravity="center"
    android:background="@drawable/card_background"
    android:layout_width="250dp"
    android:layout_height="250dp"
    >

    <TextView
        android:id="@+id/txtProduit"
        android:text="Produit"
        android:textSize="22sp"
        android:textStyle="bold"
        android:textColor="#FFFFFF"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

</LinearLayout>
```

6.3 Initialisation des composants

Le StackView utilise un **Adapter** pour afficher les données. Créer la classe **ProduitsActivity** , et faîtes appel à toutes les vues nécessaires.

```
stackView = findViewById(R.id.stackView);
txtBienvenue = findViewById(R.id.txtBienvenue);
editProduit = findViewById(R.id.editProduit);
btnAjouter = findViewById(R.id.btnAjouter);
btnLogout = findViewById(R.id.btnLogout);
```

6.4 Affichage du message de bienvenue

6.4.1 Partie théorique

On récupère le nom de l'utilisateur depuis la session.

```
txtBienvenue.setText("Bienvenue␣" + SessionManager.getUtilisateur()
    );
```

6.4.2 Partie pratique

Compléter :

```
txtBienvenue.setText("Bienvenue␣" + ____________.getUtilisateur());
```

6.5 Chargement des produits depuis le cache

6.5.1 Partie théorique

Les produits sont stockés dans une liste globale :

```
produits = CacheManager.produits;
```

6.5.2 Partie pratique

Compléter :

```
produits = ____________.produits;
```

6.6 Ajout des produits dans la liste

6.6.1 Partie théorique

On récupère les produits depuis le cache :

```
produits.add(CacheManager.cache.get("1"));
```

6.6.2 Partie pratique

Compléter :

```
produits.add(__________.cache.get("1"));
produits.add(__________.__________.get("2"));
produits.add(__________.cache.__________);
```

6.7 Association Adapter + StackView

6.7.1 Partie théorique

L'adapter permet d'afficher les données dans le StackView.

6.7.2 Partie pratique

Compléter :

```
adapter = new __________(this, produits);
stackView.set__________(adapter);
```

6.8 Ajout d'un nouveau produit

6.8.1 Partie théorique

On ajoute un produit dans la liste puis on met à jour l'affichage.

6.8.2 Partie pratique

Compléter :

```
btnAjouter.setOnClickListener(v -> {
String nouveauProduit = __________.getText().toString();

if(!nouveauProduit.isEmpty()){

    produits.__________(nouveauProduit);

    adapter.________________(); //notifier l'adapter que les donné
       es ont changé.
    ________________ //ajouter une notification d'ajout
    editProduit.________________("""); //vider le champ
}});
```

6.9 Gestion de la déconnexion

6.9.1 Partie théorique

La déconnexion consiste à :

- supprimer la session utilisateur
- revenir à l'écran principal
- fermer l'activité actuelle

```
SessionManager.logout();

Intent intent = new Intent(
    ProduitsActivity.this,
    MainActivity.class
);

startActivity(intent);
finish();
```

6.9.2 Rôle de la méthode finish()

La méthode `finish()` permet de fermer l'activité actuelle (`ProduitsActivity`) après la déconnexion. Lorsque l'utilisateur clique sur le bouton de déconnexion, une nouvelle activité (`MainActivity`) est lancée à l'aide de `startActivity(intent)`.

Cependant, sans l'appel à `finish()`, l'activité précédente reste en arrière-plan dans la pile des activités.

Cela signifie que si l'utilisateur appuie sur le bouton retour du téléphone, il pourrait revenir à l'écran des produits alors qu'il est censé être déconnecté. En utilisant `finish()`, on supprime l'activité courante de la pile, empêchant ainsi tout retour vers celle-ci et garantissant un comportement correct et sécurisé de l'application.

6.9.3 Partie pratique

Compléter :

```
btnLogout.setOnClickListener(v -> {

    SessionManager.________();

    Intent intent = new Intent(
        ProduitsActivity.this,
        ____________.class
    );

    startActivity(intent);

    ______();

});
```

7 Bonus : Ajout d'image pour chaque produit

7.1 Objectif

Dans ce bonus, nous allons permettre à l'utilisateur d'associer une image à chaque produit.

Deux types d'images seront utilisés :

— Images locales (drawable)

— Images choisies par l'utilisateur (galerie)

7.2 Partie théorique

En Android, il existe deux types d'images :

— Les images internes (ressources) : identifiées par un entier (ex : R.drawable.img1)

— Les images externes (galerie) : identifiées par un Uri

Pour gérer les deux types dans une seule liste, on utilise :

```
ArrayList<Object> images;
```

Object peut contenir :

FIGURE 2 – StackView avec Image et texte

- Integer (drawable) : On utilise int (R.drawable) pour les images locales
- Uri (galerie) : On utilise Uri pour les images uploadées

La différence entre les deux types, provient de la source de chaque image :

- R.drawable.img1 est :
  - une image déjà dans ton projet
  - compilée dans l'application
  - identifiée par un entier
- Quand l'utilisateur choisit une image, elle vient de :
  - galerie
  - stockage
  - appareil photo

  Android ne lui donne PAS un int, il genère un Uri : **adresse de l'image dans le téléphone**

  Exemple : "content ://media/external/images/media/12345"

En ligne de code, voilà comment appliquer chacune :

```
    // image locale
img.setImageResource(R.drawable.img1);

// image utilisateur
img.setImageURI(imageUri);
```

7.3 Affichage des images

Dans le constructeur de l'adapter, il faut rajouter la liste d'images.

```
public ProduitAdapter(Context context,
                      ArrayList<String> produits,
                      ArrayList<Object> images){

    this.context = context;
    this.produits = produits;
    this.images = images;
}
```

Dans l'adapter, il faut tester le type de l'image :

```
Object img = images.get(i);

if(img instanceof Integer){
    imageView.setImageResource((Integer) img);
}
else if(img instanceof Uri){
    imageView.setImageURI((Uri) img);
}
```

7.4 Récupération de l'image

```
imageUri = data.getData();
```

7.5 Choix d'une image depuis la galerie

Pour sélectionner une image, on utilise un Intent :

```
Intent intent = new Intent(Intent.ACTION_PICK);
intent.setType("image/*");
startActivityForResult(intent, 1);
```

La méthode `startActivityForResult` permet de lancer une activité et de récupérer un résultat après son exécution.

Elle est souvent utilisée pour :

— sélectionner une image depuis la galerie
— prendre une photo
— récupérer une donnée depuis une autre activité

```
@Override
protected void onActivityResult(int requestCode, int resultCode
    , Intent data) {
    super.onActivityResult(requestCode, resultCode, data);

    if(requestCode == 1 && resultCode == RESULT_OK){

        imageUri = data.getData();


    }
```

7.6 Ajout d'un produit avec image

Lors de l'ajout d'un produit, il est important d'ajouter :
— le nom dans la liste produits
— l'image dans la liste images

```
produits.add(nouveauProduit);

if(imageUri != null){
    images.add(imageUri);
}else{
    images.add(R.drawable.img1);
}
```

7.7 Important

Les deux listes doivent toujours avoir la même taille.
Sinon, l'application peut provoquer une erreur :

```
IndexOutOfBoundsException
```

7.8 Question de réflexion

1. Pourquoi utilise-t-on Uri pour les images choisies par l'utilisateur ?
2. Pourquoi ne peut-on pas utiliser R.drawable pour une image de la galerie ?
3. Que se passe-t-il si les listes produits et images n'ont pas la même taille ?

8 Mise en cache des données (JSON et Images)

8.1 Introduction

Dans une application mobile, le téléchargement des données depuis Internet peut être lent et coûteux en ressources.
Le **cache** permet de stocker temporairement des données afin de :
— améliorer les performances
— réduire les appels réseau
— accélérer l'affichage

8.2 Types de données mises en cache

Dans ce TP, nous utilisons deux types de cache :
— Cache JSON : stockage sous forme de chaîne de caractères (String)
— Cache image : stockage sous forme de Bitmap (image en mémoire)

8.3 Cache JSON

Le JSON est stocké dans une variable :

```
public static String jsonCache;
```

Avant de télécharger les données, on vérifie si elles existent déjà :

```
if(CacheManager.jsonCache != null){
    txtJson.setText(CacheManager.jsonCache);
}
```

Sinon, on effectue un téléchargement depuis Internet.

8.4 Cache des images (Bitmap)

Une image téléchargée est convertie en **Bitmap** :

```
Bitmap bitmap = BitmapFactory.decodeStream(inputStream);
```

Le Bitmap est ensuite stocké dans un cache mémoire :

```
CacheManager.imageCache.put("img1", bitmap);
```

Pour réutiliser l'image :

```
Bitmap bmp = CacheManager.imageCache.get("img1");
imageView.setImageBitmap(bmp);
```

8.5 Pourquoi utiliser Bitmap ?

Un Bitmap représente une image sous forme de pixels en mémoire.
Cela permet :
— un affichage rapide
— d'éviter la reconversion de l'image
— d'améliorer la fluidité

8.6 Fonctionnement du cache

Le fonctionnement du cache suit les étapes suivantes :

1. Vérifier si la donnée existe dans le cache
2. Si oui : l'utiliser directement
3. Sinon : télécharger la donnée
4. Stocker la donnée dans le cache
5. Afficher la donnée

8.7 Limites du cache

Le cache utilisé dans ce TP est stocké en mémoire RAM.
Cela implique :
— les données sont temporaires
— elles sont perdues si l'application est fermée
— la taille du cache est limitée

8.8 Optimisation des images

Les images peuvent être volumineuses en mémoire.

Il est possible de réduire leur taille lors du chargement :

```
BitmapFactory.Options options = new BitmapFactory.Options();
options.inSampleSize = 2;

Bitmap bitmap = BitmapFactory.decodeStream(inputStream, null,
    options);
```

Cela permet de :

— réduire la consommation mémoire

— éviter les erreurs de type OutOfMemory

8.9 Threads

On utilise un Thread pour éviter de bloquer l'interface utilisateur lors des opérations réseau.

Le cache JSON stocke du texte (String), tandis que le cache d'image stocke des Bitmaps (images en mémoire). Mais si l'application est fermée, les données en cache (RAM) sont perdues.

8.10 Code

8.10.1 CacheManager

```
    package com.example.tp13_cache;

import android.graphics.Bitmap;
import android.util.LruCache;

import java.util.ArrayList;

public class CacheManager {

    public static LruCache<String,String> cache;
    public static ArrayList<String> produits = new ArrayList<>();

    public static LruCache<String, Bitmap> imageCache;
    public static String jsonCache;

    public static void init(){
        if(imageCache == null){
        int cacheSize = 4 * 1024 * 1024;

        imageCache = new LruCache<>(cacheSize);
    }}

    public static void initCache(){

        if(cache == null){
```

```
            int cacheSize = 4 * 1024 * 1024;

            cache = new LruCache<>(cacheSize);

            cache.put("1","Chaussures");
            cache.put("2","T-shirt");
            cache.put("3","Pantalon");
            cache.put("4","Casquette");
            cache.put("5","Montre");

        }
    }
    public static void clear(){
        jsonCache = null;
        if(imageCache != null){
            imageCache.evictAll();
        }
    }

}
```

8.10.2 CacheActivity

```
    package com.example.tp13_cache;

import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.os.Bundle;
import android.widget.Button;
import android.widget.ImageView;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

import java.io.BufferedReader;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.net.URL;

public class CacheActivity extends AppCompatActivity {

    TextView txtJson;
    ImageView imageView;

    Button btnLoadJson, btnLoadImage, btnClearCache;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_cache);
```

```java
        txtJson = findViewById(R.id.txtJson);
        imageView = findViewById(R.id.imageView);

        btnLoadJson = findViewById(R.id.btnLoadJson);
        btnLoadImage = findViewById(R.id.btnLoadImage);
        btnClearCache = findViewById(R.id.btnClearCache);

        CacheManager.init();


        //        Bouton JSON
        btnLoadJson.setOnClickListener(v -> loadJson());

        //        Bouton Image
        btnLoadImage.setOnClickListener(v -> loadImage());

        //        Bouton vider cache
        btnClearCache.setOnClickListener(v -> {
            CacheManager.clear();
            txtJson.setText("Cache␣vidé");
            imageView.setImageBitmap(null);
        });
    }

    //        JSON
    private void loadJson(){

        if(CacheManager.jsonCache != null){
            txtJson.setText("   ␣CACHE\n\n" + CacheManager.
               jsonCache);
            return;
        }

        new Thread(() -> {

            try {
                URL url = new URL("https://jsonplaceholder.typicode
                   .com/posts/1");

                BufferedReader reader = new BufferedReader(
                        new InputStreamReader(url.openStream())
                );

                StringBuilder result = new StringBuilder();
                String line;

                while((line = reader.readLine()) != null){
                    result.append(line);
                }
```

```
                CacheManager.jsonCache = result.toString();

                runOnUiThread(() ->
                        txtJson.setText("      ␣INTERNET\n\n" +
                           CacheManager.jsonCache)
                );

            } catch (Exception e){
                e.printStackTrace();
            }

        }).start();
    }

    //        IMAGE
    private void loadImage(){

        Bitmap cached = CacheManager.imageCache.get("img1");

        if(cached != null){
            imageView.setImageBitmap(cached);
            return;
        }

        new Thread(() -> {

            try {
                InputStream input = new URL("https://picsum.photos
                   /300").openStream();

                Bitmap bitmap = BitmapFactory.decodeStream(input);

                CacheManager.imageCache.put("img1", bitmap);

                runOnUiThread(() ->
                        imageView.setImageBitmap(bitmap)
                );

            } catch (Exception e){
                e.printStackTrace();
            }

        }).start();
    }
}
```

8.11 Conclusion

Le cache est une technique essentielle en développement mobile.
Il permet :

— d'améliorer les performances

— de réduire les temps de chargement
— d'offrir une meilleure expérience utilisateur

Lien vers Projet Android : TP13

9 Questions de réflexion

1. Quelle est la différence entre mémoire volatile et stockage SQLite ?
2. Pourquoi utiliser un cache dans une application mobile ?
3. Que se passe-t-il si l'application est fermée lorsque les données sont uniquement en RAM ?
4. Dans quel cas utiliser une session utilisateur ?